

Floating Point Arithmetic

Recall the fundamental expression for binary representation of an N-bit (non-negative) Boolean value:

$$b_{N-1} \dots b_1 b_0 = b_{N-1} \times 2^{N-1} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

Using 2s-complement form, the most significant bit is given a negative weight, but everything else stays the same.

We can use a similar scheme to represent fractional quantities by moving the “decimal point” to a new location. Suppose, for example, that we decide to represent an 8-bit number with three bits of fraction. Then we would have:

$$b_4b_3b_2b_1b_0b_{-1}b_{-2}b_{-3} = b_4 \times 2^4 + \dots + b_0 \times 2^0 + b_{-1} \times 2^{-1} + \dots$$

This scheme is referred to as **fixed point notation**. With fixed point notation, we can use standard integer arithmetic operations to perform arithmetic on our non-integer numbers.

Examples (Using 8-bit numbers with 3 bits of fraction):

Addition:

$$4.625 + 2.75 = 7.375 \quad 00100101 + 00010110 = 00111011$$

Subtraction:

$$2.75 - 4.625 = -1.875 \quad 00010110 + 11011011 = 11110001$$

For addition and subtraction, the “decimal point” does not move.

For multiplication, we need to adjust the decimal point (shift) to retain our fixed point notation. Shifting will cost us precision:

$$\begin{aligned} 2.75 \times 4.625 &= 12.71875 \\ 00010110 \times 00100101 &= 0000001100101110 \\ &\approx 01100101 \\ &= 12.625 \end{aligned}$$

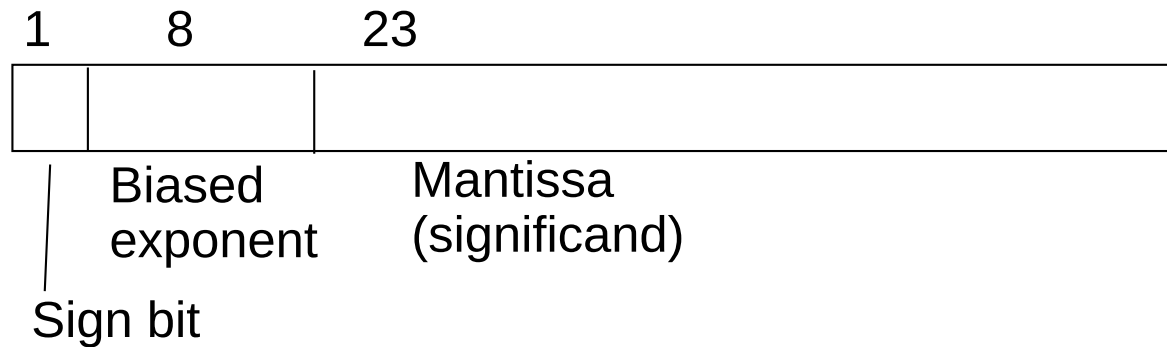
Fixed-point notation is simple to implement (in fact, it requires no hardware support), but it is very limited in both range and accuracy. To overcome these problems, a special arithmetic notation is used for fractional quantities called **floating-point notation**.

Floating point notation is similar to **scientific notation** we use all the time. Essentially, we use a **significand-base-exponent** scheme to expand the range of our notation:

4.94×10^{299} , Significand = 4.94, Base = 10, Exponent = 299

4.94×10^{-130} , Significand = 4.94, Base = 10, Exponent = -130

A typical 32-bit floating point format looks like this:



Sign bit 1 indicates the number is negative. 0 indicates the number is positive

Biased exponent The k -bit exponent is stored in a “biased format”. This means $E = BE - (2^{k-1} - 1)$. This is done so that the biased exponents can be compared **as though they were unsigned**.

Mantissa The mantissa (significand) contains the “significant digits” for the number. The mantissa is **normalized**, which means that the most significant bit (MSB) is **assumed to be 1** and is **not stored**.

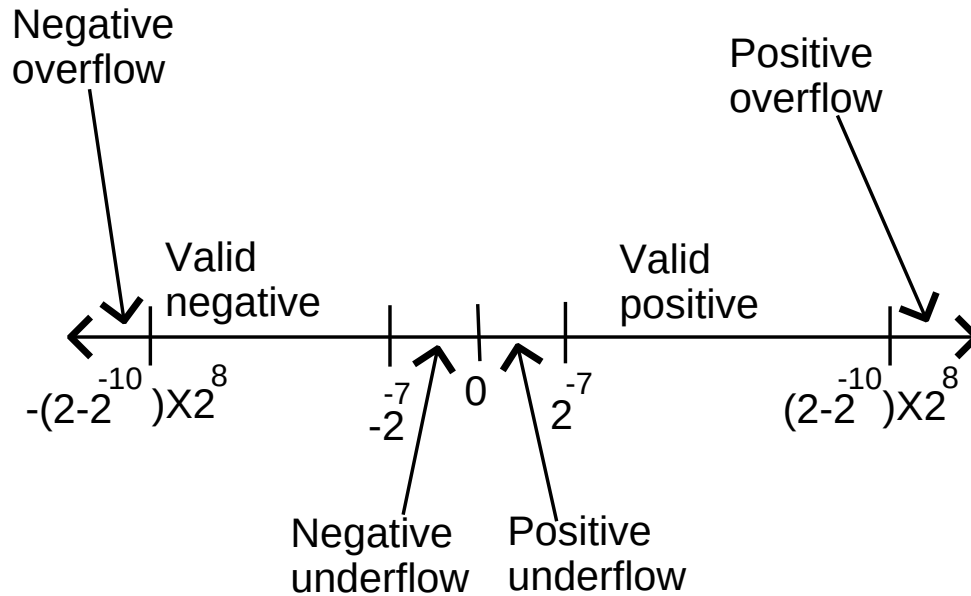
To make this clearer, assume a 16-bit floating point format with 4 exponent bits. The number will be stored as:

$$SE_3E_2E_1E_0B_{-1}B_{-2}B_{-3}B_{-4}B_{-5}B_{-6}B_{-7}B_{-8}B_{-9}B_{-10}$$

And it will represent:

$$N = (-1)^S \times 1.B_{-1}B_{-2} \dots B_{-10} \times 2^{E-7}$$

Here is the number line for our proposed floating point scheme:



Interesting points:

- The “farther away from zero” you get, the less dense the representation becomes.
- There is no such thing as “exactly zero”. In most representations, a bit pattern is set aside to mean zero.

Special numbers in IEEE floating point notation

All the special values below remove possibilities from the representation.

Zero An exponent of zero and a fraction of zero are positive and negative zero.

+/- Infinity An exponent of all ones and a fraction of zero are positive and negative infinity.

De-normalized An exponent of zero and a non-zero fraction is a **denormalized number**. Denormalized numbers can be used to represent numbers in the exponent underflow range.

NaN An exponent of all ones and a non-zero fraction is called NaN (Not a Number). It is typically used to signal exceptional conditions. Examples of NaN results are $\infty \times \infty$, $0/0$, $\sqrt{(x)} \mid x < 0$ and so on.

Analysis Problem

Represent the numbers $1/3$, 500000 and π in our 16-bit floating point scheme.

Arithmetic with floating-point numbers

Addition and subtraction are four-phase processes:

Zero check The first step changes one sign (since adding negatives is like subtraction). Then, if one number is zero, return the other unchanged.

Alignment In order to add or subtract floating point numbers, it is necessary to align the decimal points. The smaller (smaller exponent) is shifted right (and the exponent is increased) until the exponents match. If there is underflow (the smaller number reaches zero), the process terminates.

Addition Add the significands. If necessary, add to the common exponent and shift right.

Normalization Shift left to find a non-zero digit, updating the exponent.

Oddly enough, multiplication and division are **simpler** for floating point values. To multiply:

Zero check If one operand is zero, we're finished, so stop.

Add exponents The exponents are simply added, with care to handle the biasing properly.

Multiply The mantissae can be multiplied as though they were integers.

Normalize The result is normalized and rounded.

To divide:

Zero check If the divisor is zero, something is wrong. Some systems will report failure (interrupt). Others simply set the result to infinity. If the dividend is zero, so is the result.

Subtract exponents The exponents are simply subtracted, with care to handle the biasing properly.

Divide The mantissae are divided as though they were integers.

Normalize The result is normalized and rounded.

Rounding

The result of any floating point operation must be rounded to have the correct number of significant figures. Common schemes:

Round to nearest Find the “nearest number” that can be represented. Essentially, this means if the first un-represented bit is 1, round up. If the first un-represented bit is 0, round down. If the un-represented bits are 10000000....0 (it’s exactly half-way), make it even.

Round to $\pm$ infinity Some systems use **interval arithmetic**, which finds the upper and lower bounds that bracket the correct result. The narrowness of this range tells the user if the result is “close enough” or not.

Round to zero Just truncate the left-over bits. This scheme is the simplest, but contains a bias.